Лабораторная работа №4

Продвинутое использование git.

Лопатченко Полина Андреевна

Содержание

1 Цель работы **5**
1.1 Рабочий процесс Gitflow 5
1.2 Семантическое версионирование 8
1.3 Общепринятые коммиты 10

2 Выполнение лабораторной работы **14**
2.1 Установка программного обеспечения 14
2.2 Практический сценарий использования git 16

3 Выводы **20**

Список иллюстраций

2.1 Установка git-flow 14
2.2 Установка Node.js 15
2.3 Запуск pnpm setup и выполнение source ~/.bashrc 15
2.4 Создание репозитория GitHub 16
2.5 Создание первого коммита и его загрузка на github 16
2.6 Выполнение конфигурации пакетов Node.js 17
2.7 Заполнение нескольких параметров пакета и конфигурация формата коммитов 17
2.8 Инициализация git-flow 18
2.9 Проверка нахождения на ветке develop, загрузка всего репозитория в хранилище, установка внешней ветки как вышестоящей для этой ветки и создание релиза с версией 1.0.0 18
2.10 Создание релиза с версией `1.2.3`, обновление номера версии в `package.json`, создание журнала изменений и добавление журнал изменений в индекс 19
2.11 Отправка данных на github, создание релиза на github с комментариями из журнала изменений 19

Список таблиц

1 Цель работы

Получение навыков правильной работы с репозиториями git. # Задание Научиться работать с git на более высоком уровне. # Теоретическое введение

1.1 Рабочий процесс Gitflow

- Рабочий процесс *Gitflow Workflow*. Будем описывать его с использованием пакета `git-flow`.

1.1.1 Общая информация

- Gitflow Workflow опубликована и популяризована Винсентом Дриссеном.
- Gitflow Workflow предполагает выстраивание строгой модели ветвления с учётом выпуска проекта.
- Данная модель отлично подходит для организации рабочего процесса на основе релизов.
- Работа по модели Gitflow включает создание отдельной ветки для исправлений ошибок в рабочей среде.
- Последовательность действий при работе по модели Gitflow: - Из ветки `master` создаётся ветка `develop`. - Из ветки `develop` создаётся ветка `release`. - Из ветки `develop` создаются ветки `feature`. - Когда работа над веткой `feature` завершена, она сливается с веткой `develop`. - Когда работа над веткой релиза `release` завершена, она сливается в ветки `develop` и `master`.

- Если в `master` обнаружена проблема, из `master` создаётся ветка `hotfix`. - Когда работа над веткой исправления `hotfix` завершена, она сливается в ветки `develop` и `master`.

1.1.2 Процесс работы с Gitflow

1. Основные ветки (`master`) и ветки разработки (`develop`)

```
- Для фиксации истории проекта в рамках этого процесса вместо одной ветки `maste
```

   - При использовании библиотеки расширений `git-flow` нужно инициализировать структуру в существующем репозитории:

     ```
     git flow init
     ```

     - Для github параметр `Version tag prefix` следует установить в `v`.
     - После этого проверьте, на какой ветке Вы находитесь:

     ```
     git branch
     ```

2. Функциональные ветки (feature)

   - Под каждую новую функцию должна быть отведена собственная ветка, которую можно отправлять в центральный репозиторий для создания резервной копии или совместной работы команды. Ветки `feature` создаются не на основе `master`, а на основе `develop`. Когда работа над функцией завершается, соответствующая ветка сливается обратно с веткой `develop`. Функции не следует отправлять напрямую в ветку `master`.
   - Как правило, ветки `feature` создаются на основе последней ветки `develop`.

     1. Создание функциональной ветки

        - Создадим новую функциональную ветку:

```
git flow feature start feature_branch
```

- Создадим новую функциональную ветку:

```
git flow feature start feature_branch
```

- Далее работаем как обычно.

2. Окончание работы с функциональной веткой
   - По завершении работы над функцией следует объединить ветку `feature_branch` с `develop`:

```
git flow feature finish feature_branch
```

3. Ветки выпуска (release)
   - Когда в ветке `develop` оказывается достаточно функций для выпуска, из ветки `develop` создаётся ветка `release`. Создание этой ветки запускает следующий цикл выпуска, и с этого момента новые функции добавить больше нельзя — допускается лишь отладка, создание документации и решение других задач. Когда подготовка релиза завершается, ветка `release` сливается с `master` и ей присваивается номер версии. После нужно выполнить слияние с веткой `develop`, в которой с момента создания ветки релиза могли возникнуть изменения.
   - Благодаря тому, что для подготовки выпусков используется специальная ветка, одна команда может дорабатывать текущий выпуск, в то время как другая команда продолжает работу над функциями для следующего.
   - Создать новую ветку release можно с помощью следующей команды:

```
git flow release start 1.0.0
```

- Для завершения работы на ветке release используются следующие команды:

```
git flow release finish 1.0.0
```

4. Ветки исправления (hotfix)
   - Ветки поддержки или ветки `hotfix` используются для быстрого внесения исправлений в рабочие релизы. Они создаются от ветки `master`. Это единственная ветка, которая должна быть создана непосредственно от `master`. Как только исправление завершено, ветку следует объединить с `master` и `develop`. Ветка `master` должна быть помечена обновлённым номером версии.
   - Наличие специальной ветки для исправления ошибок позволяет команде решать проблемы, не прерывая остальную часть рабочего процесса и не ожидая следующего цикла релиза.
   - Ветку `hotfix` можно создать с помощью следующих команд:

   ```
   git flow hotfix start hotfix_branch
   ```

   По завершении работы ветка `hotfix` объединяется с `master` и `develop`: `git flow hotfix finish hotfix_branch`

1.2 Семантическое версионирование

Семантический подход в версионированию программного обеспечения.

1.2.1 Краткое описание семантического версионирования

- Семантическое версионирование описывается в манифесте семантического версионирования.
- Кратко его можно описать следующим образом:
  - Версия задаётся в виде кортежа `МАЖОРНАЯ_ВЕРСИЯ.МИНОРНАЯ_ВЕРСИЯ.ПАТЧ`.

- Номер версии следует увеличивать:
  * МАЖОРНУЮ версию, когда сделаны обратно несовместимые изменения API.
  * МИНОРНУЮ версию, когда вы добавляете новую функциональность, не нарушая обратной совместимости.
  * ПАТЧ-версию, когда вы делаете обратно совместимые исправления.
- Дополнительные обозначения для предрелизных и билд-метаданных возможны как дополнения к МАЖОРНАЯ.МИНОРНАЯ.ПАТЧ формату.

1.2.2 Программное обеспечение

- Для реализации семантического версионирования создано несколько программных продуктов.
- При этом лучше всего использовать комплексные продукты, которые используют информацию из коммитов системы версионирования.
- Коммиты должны иметь стандартизованный вид.
- В семантическое версионирование применяется вместе с *общепринятыми коммитами*.

  1. Пакет Conventional Changelog
     - Пакет Conventional Changelog является комплексным решением по управлению коммитами и генерации журнала изменений.
     - Содержит набор утилит, которые можно использовать по-отдельности.

1.3 Общепринятые коммиты

Использование спецификации Conventional Commits.

1.3.1 Описание

Спецификация Conventional Commits:

- Соглашение о том, как нужно писать сообщения commit'ов.
- Совместимо с SemVer. Даже вернее сказать, сильно связано с семантическим версионированием.
- Регламентирует структуру и основные типы коммитов.

  1. Структура коммита "' ():

     Или, по-русски:

     (): [необязательное тело] [необязательный нижний колонтитул] "'

     - Заголовок является обязательным.
     - Любая строка сообщения о фиксации не может быть длиннее 100 символов.
     - Тема (subject) содержит краткое описание изменения.
       * Используйте повелительное наклонение в настоящем времени: «изменить» («change» not «changed» nor «changes»).
       * Не используйте заглавную первую букву.
       * Не ставьте точку в конце.
     - Тело (body) должно включать мотивацию к изменению и противопоставлять это предыдущему поведению.
       * Как и в теме, используйте повелительное наклонение в настоящем времени.

- Нижний колонтитул (footer) должен содержать любую информацию о критических изменениях.
  * Следует использовать для указания внешних ссылок, контекста коммита или другой мета информации.
  * Также содержит ссылку на issue (например, на github), который закрывает эта фиксация.
  * Критические изменения должны начинаться со слова `BREAKING CHANGE`: с пробела или двух символов новой строки. Затем для этого используется остальная часть сообщения фиксации.

2. Типы коммитов

   1. Базовые типы коммитов
      - `fix:` — коммит типа fix исправляет ошибку (bug) в вашем коде (он соответствует PATCH в SemVer).
      - `feat:` — коммит типа feat добавляет новую функцию (feature) в ваш код (он соответствует MINOR в SemVer).
      - `BREAKING CHANGE:` — коммит, который содержит текст `BREAKING CHANGE:` в начале своего не обязательного тела сообщения (body) или в подвале (footer), добавляет изменения, нарушающие обратную совместимость вашего API (он соответствует MAJOR в SemVer). BREAKING CHANGE может быть частью коммита любого типа.
      - `revert:` — если фиксация отменяет предыдущую фиксацию. Начинается с revert:, за которым следует заголовок отменённой фиксации. В теле должно быть написано: Это отменяет фиксацию (это SHA-хэш отменяемой фиксации).
      - Другое: коммиты с типами, которые отличаются от `fix:` и `feat:`, также разрешены. Например, **commitlint/config-conventional**

(основанный на The Angular convention) рекомендует: chore:, docs:, style:, refactor:, perf:, test:, и другие.

2. Соглашения The Angular convention
   - Одно из популярных соглашений о поддержке исходных кодов — конвенция Angular (The Angular convention).
   1. Типы коммитов The Angular convention Конвенция Angular (The Angular convention) требует следующие типы коммитов:
   - `build:` — изменения, влияющие на систему сборки или внешние зависимости (примеры областей (scope): gulp, broccoli, npm).
   - `ci:` — изменения в файлах конфигурации и скриптах CI (примеры областей: Travis, Circle, BrowserStack, SauceLabs).
   - `docs:` — изменения только в документации.
   - `feat:` — новая функция.
   - `fix:` — исправление ошибок.
   - `perf:` — изменение кода, улучшающее производительность.
   - `refactor:` — Изменение кода, которое не исправляет ошибку и не добавляет функции (рефакторинг кода).
   - `style:` — изменения, не влияющие на смысл кода (пробелы, форматирование, отсутствие точек с запятой и т. д.).
   - `test:` — добавление недостающих тестов или исправление существующих тестов.
   1. Области действия (scope) Областью действия должно быть имя затронутого пакета npm (как его воспринимает человек, читающий журнал изменений, созданный из сообщений фиксации). Есть несколько исключений из правила «использовать имя пакета»:
   - `packaging` — используется для изменений, которые изменяют структуру пакета, например, изменения общедоступного

пути.

  - `changelog` — используется для обновления примечаний к выпуску в CHANGELOG.md.
  - отсутствует область действия — полезно для изменений стиля, тестирования и рефакторинга, которые выполняются во всех пакетах (например, style: добавить отсутствующие точки с запятой).

3. Соглашения **commitlint/config-conventional** Соглашение **commitlint/config-conventional** входит в пакет Conventional Changelog. В целом в этом соглашении придерживаются соглашения Angular.

2 Выполнение лабораторной работы

2.1 Установка программного обеспечения

2.1.1 Установка git-flow

Установила git-flow (см. рис.1).

Рисунок 2.1: Установка git-flow

2.1.2 Установка Node.js

Установила Node.js (см. рис.2).

```
palopatchenko@palopatchenko:~$ sudo dnf install nodejs
Updating and loading repositories:
Repositories loaded.
Package                                    Arch       Version
Installing:
 nodejs                                    x86_64     1:22.22.0-2.fc43
Installing dependencies:
 nodejs-libs                               x86_64     1:22.22.0-2.fc43
Installing weak dependencies:
 nodejs-docs                               noarch     1:22.22.0-2.fc43
 nodejs-full-i18n                          x86_64     1:22.22.0-2.fc43
 nodejs-npm                                x86_64     1:10.9.4-1.22.22.0.2.

Transaction Summary:
 Installing:         5 packages

Total size of inbound packages is 40 MiB. Need to download 40 MiB.
After this operation, 215 MiB extra will be used (install 215 MiB, remove
Is this ok [y/N]: y
[1/5] nodejs-1:22.22.0-2.fc43.x86_64
[2/5] nodejs-libs-1:22.22.0-2.fc43.x86_64                          0% [
[3/5] nodejs-docs-1:22.22.0-2.fc43.noarch                          0% [
[4/5] nodejs-npm-1:10.9.4-1.22.22.0.2.fc43.x86_64                  0% [
```

Рисунок 2.2: Установка Node.js

2.1.3 Настройка Node.js

Запустила `pnpm setup` и выполнила `source ~/.bashrc` (см. рис.3).

```
palopatchenko@palopatchenko:~$ pnpm setup
Appended new lines to /home/palopatchenko/.bashrc

Next configuration changes were made:
export PNPM_HOME="/home/palopatchenko/.local/share/pnpm"
case ":$PATH:" in
  *":$PNPM_HOME:"*) ;;
  *) export PATH="$PNPM_HOME:$PATH" ;;
esac

To start using pnpm, run:
source /home/palopatchenko/.bashrc
palopatchenko@palopatchenko:~$ source ~/.bashrc
```

Рисунок 2.3: Запуск pnpm setup и выполнение source ~/.bashrc

2.2 Практический сценарий использования git

2.2.1 Создание репозитория git

1. Подключение репозитория к github

- Создала репозиторий на GitHub. Назвала его `git-extended` (см. рис.4).

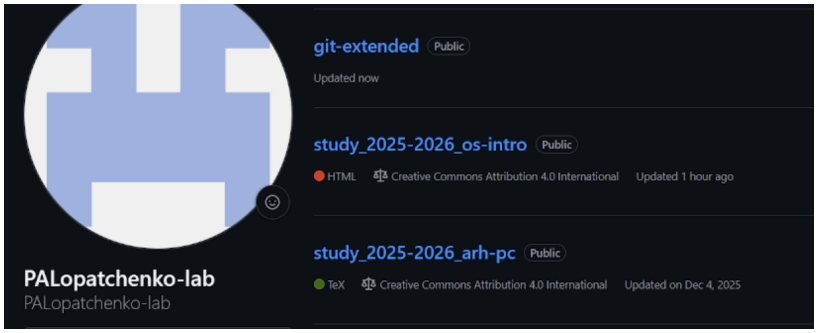

Рисунок 2.4: Создание репозитория GitHub

- Сделала первый коммит и выложила его на github (см. рис.5).

Рисунок 2.5: Создание первого коммита и его загрузка на github

1. Конфигурация общепринятых коммитов (см. рис.6).

- Конфигурация пакетов Node.js (см. рис.6).

```
palopatchenko@palopatchenko:~/git-extended$ pnpm init
Wrote to /home/palopatchenko/git-extended/package.json

{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "packageManager": "pnpm@10.27.0"
```

Рисунок 2.6: Выполнение конфигурации пакетов Node.js

Заполнила несколько параметров пакета и сконфигурировала формат коммитов (см. рис.7).

```
  GNU nano 8.5                                          package.jso
{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "Git repo for educational purposes",
  "main": "index.js",
  "repsitory": "git@github.com:PALopatchenko-lab/git-extended.git"
  "author": "Polina Lopatchenko lopatapol228@gmail.com",
  "license": "CC-BY-4.0",
  "config": {
    "commitizen": {
      "path": "cz-conventional-changelog"
    }
  }
}
```

Рисунок 2.7: Заполнение нескольких параметров пакета и конфигурация формата коммитов

1. Конфигурация git-flow

- Инициализировал git-flow (см. рис.11).

```
palopatchenko@palopatchenko:~/git-extended$ git flow init

Which branch should be used for bringing forth production releases?
   - main
Branch name for production releases: [main]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
```

Рисунок 2.8: Инициализация git-flow

- Проверила, что я на ветке `develop`, загрузила весь репозиторий в хранилище, установила внешнюю ветку как вышестоящую для этой ветки и создала релиз с версией 1.0.0 (см. рис.12).

```
palopatchenko@palopatchenko:~/git-extended$ git branch
* develop
  main
palopatchenko@palopatchenko:~/git-extended$ git status
On branch develop
nothing to commit, working tree clean
palopatchenko@palopatchenko:~/git-extended$ git push --all
Enter passphrase for key '/home/palopatchenko/.ssh/id_ed25519':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/PALopatchenko-lab/git-extended/pull/new/d
remote:
To github.com:PALopatchenko-lab/git-extended.git
 * [new branch]      develop -> develop
palopatchenko@palopatchenko:~/git-extended$ git branch --set-upstream-to=
branch 'develop' set up to track 'origin/develop'.
```

Рисунок 2.9: Проверка нахождения на ветке develop, загрузка всего репозитория в хранилище, установка внешней ветки как вышестоящей для этой ветки и создание релиза с версией 1.0.0

2.2.2 Работа с репозиторием git

1. Создание релиза git-flow

- Создала релиз с версией `1.2.3`, обновила номер версии в `package.json`, создала журнал изменений и добавила журнал изменений в индекс, залила релизную ветку в основную ветку, (см. рис.16).

```
palopatchenko@palopatchenko:~/git-extended$ git flow release start 1.2.3
Branches 'develop' and 'origin/develop' have diverged.
And local branch 'develop' is ahead of 'origin/develop'.
Switched to a new branch 'release/1.2.3'

Summary of actions:
- A new branch 'release/1.2.3' was created, based on 'develop'
- You are now on branch 'release/1.2.3'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

     git flow release finish '1.2.3'

palopatchenko@palopatchenko:~/git-extended$ ls
CHANGELOG.md  first  package.json
palopatchenko@palopatchenko:~/git-extended$ nano package.json
palopatchenko@palopatchenko:~/git-extended$ standard-changelog
palopatchenko@palopatchenko:~/git-extended$ git add CHANGELOG.md
palopatchenko@palopatchenko:~/git-extended$ git commit -am "chore(site): update changelog"
[release/1.2.3 a763197] chore(site): update changelog
 2 files changed, 3 insertions(+), 1 deletion(-)
palopatchenko@palopatchenko:~/git-extended$ git flow release finish 1.2.3
Branches 'develop' and 'origin/develop' have diverged.
And local branch 'develop' is ahead of 'origin/develop'.
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
Merge made by the 'ort' strategy.
 CHANGELOG.md | 2 ++
```

Рисунок 2.10: Создание релиза с версией `1.2.3`, обновление номера версии в `package.json`, создание журнала изменений и добавление журнал изменений в индекс

- Отправила данные на github, создала релиз на github с комментариями из журнала изменений (см. рис.17).

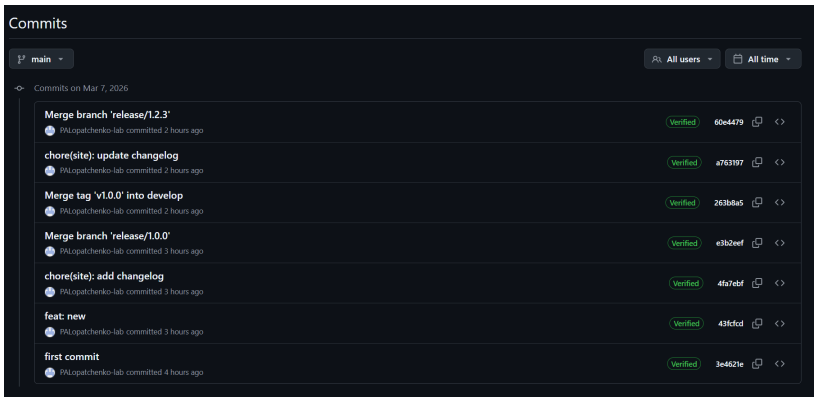

Рисунок 2.11: Отправка данных на github, создание релиза на github с комментариями из журнала изменений

3 Выводы

В ходе данной лабораторной работы были получены навыки правильной работы с репозиториями git.